```
#include <iostream>
using namespace std;

#define INF 99999

int cost[20][20];
int n;

void prim()
{
    int visited[20] = {0};
    int edges = 0;
    int totalCost = 0;

    visited[0] = 1;

    cout << "\nSelected edges:\n";

    while (edges < n - 1)
    {
        int minCost = INF;
        int u = -1, v = -1;

        for (int i = 0; i < n; i++)
        {
            if (visited[i])
            {
                for (int j = 0; j < n; j++)
                {
                    if (!visited[j] &&
```

```
                cost[i][j] < minCost)
            {
                minCost = cost[i][j];
                u = i;
                v = j;
            }
        }
    }
}

if (u == -1)
{
    cout << "\nGraph is disconnected.\n";
    cout << "Spanning tree cannot be formed.\n";
    return;
}

visited[v] = 1;
edges++;
totalCost += minCost;

cout << "Office " << u + 1
     << " - Office " << v + 1
     << " | Weight = " << minCost
     << " | Running Total = "
     << totalCost << endl;
}

cout << "\nSpanning tree can be formed.\n";
cout << "Minimum total cable cost = "
```

```
        << totalCost << endl;
}

int main()
{
    cout << "Enter number of offices: ";
    cin >> n;

    if (n < 1 || n > 20)
    {
        cout << "Invalid number of offices.\n";
        return 0;
    }

    cout << "Enter cost matrix (0 for no link):\n";

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            cin >> cost[i][j];

            if (i != j && cost[i][j] == 0)
                cost[i][j] = INF;
        }
    }

    cout << "\nOriginal network:";
    prim();
```

```
    int u, v, newWeight;

    cout << "\nEnter offices for edge update: ";
    cin >> u >> v;

    cout << "Enter new edge weight: ";
    cin >> newWeight;

    if (u < 1 || u > n ||
        v < 1 || v > n ||
        u == v || newWeight < 0)
    {
        cout << "Invalid edge or weight.\n";
        return 0;
    }

    cost[u - 1][v - 1] = newWeight;
    cost[v - 1][u - 1] = newWeight;

    cout << "\nUpdated network:";
    prim();

    return 0;
}Sample Input
#include <iostream>
using namespace std;

#define INF 99999

int cost[20][20];
```

```
int n;

void prim()
{
    int visited[20] = {0};
    int edges = 0;
    int totalCost = 0;

    visited[0] = 1;

    cout << "\nSelected edges:\n";

    while (edges < n - 1)
    {
        int minCost = INF;
        int u = -1, v = -1;

        for (int i = 0; i < n; i++)
        {
            if (visited[i])
            {
                for (int j = 0; j < n; j++)
                {
                    if (!visited[j] &&
                        cost[i][j] < minCost)
                    {
                        minCost = cost[i][j];
                        u = i;
                        v = j;
                    }
```

```
            }
        }
    }

    if (u == -1)
    {
        cout << "\nGraph is disconnected.\n";
        cout << "Spanning tree cannot be formed.\n";
        return;
    }

    visited[v] = 1;
    edges++;
    totalCost += minCost;

    cout << "Office " << u + 1
         << " - Office " << v + 1
         << " | Weight = " << minCost
         << " | Running Total = "
         << totalCost << endl;
  }

  cout << "\nSpanning tree can be formed.\n";
  cout << "Minimum total cable cost = "
       << totalCost << endl;
}

int main()
{
  cout << "Enter number of offices: ";
```

```
cin >> n;

if (n < 1 || n > 20)
{
    cout << "Invalid number of offices.\n";
    return 0;
}

cout << "Enter cost matrix (0 for no link):\n";

for (int i = 0; i < n; i++)
{
    for (int j = 0; j < n; j++)
    {
        cin >> cost[i][j];

        if (i != j && cost[i][j] == 0)
            cost[i][j] = INF;
    }
}

cout << "\nOriginal network:";
prim();

int u, v, newWeight;

cout << "\nEnter offices for edge update: ";
cin >> u >> v;

cout << "Enter new edge weight: ";
```

```
    cin >> newWeight;

    if (u < 1 || u > n ||
        v < 1 || v > n ||
        u == v || newWeight < 0)
    {
        cout << "Invalid edge or weight.\n";
        return 0;
    }

    cost[u - 1][v - 1] = newWeight;
    cost[v - 1][u - 1] = newWeight;

    cout << "\nUpdated network:";
    prim();

    return 0;
}
```